

Modelling Data and Advanced Mongoose

Locations (Geospatial Data)

Sy: 1

```
6  const tourSchema = new mongoose.Schema(  
7    {  
92    startLocation: {  
93      // GeoJSON  
94      type: {  
95        type: String,  
96        default: 'Point',  
97        enum: ['Point'],  
98      },  
99      coordinates: [Number],  
100     address: String,  
101     description: String,  
102   },  
103   locations: [  
104     {  
105       type: {  
106         type: String,  
107         default: 'Point',  
108         enum: ['Point'],  
109       },  
110       coordinates: [Number],  
111       address: String,  
112       description: String,  
113       day: Number,  
114     },  
115   ],  
116   guides: [  
117     {  
118       type: mongoose.Schema.ObjectId,  
119       ref: 'User',  
120     },  
121   ],  
122 },  
127 );
```

Start Location için kullanılan **Geojson** veri tipi.

Locations dizisi için de **Geojson** veri tipi kullanılmakta.

Modelling Data and Advanced Mongoose

Populating 1

Sy: 2

tourSchema.guides Property

```
6 const tourSchema = new mongoose.Schema({
7   {
8     // ...
9     // ...
10    // ...
11    // ...
12    // ...
13    // ...
14    // ...
15    // ...
16    guides: [
17      {
18        type: mongoose.Schema.ObjectId,
19        ref: 'User',
20      },
21    ],
22  },
23 });
```

tourSchema.find populate

```
1 tourSchema.pre(/^find/, function (next) {
2   this.populate({
3     path: 'guides',
4     select: '-__v -passwordChangedAt',
5   });
6   next();
7 });
```

Get Tour

GET {{URL}} api/v1/tours/5c88fa8cf4afda39709c2951

Params Authorization Headers (9) Body Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies (1) Headers (21) Test Results Visualize

```
{
  "secretTour": false,
  "guides": [
    {
      "role": "guide",
      "_id": "682cb8aaad40925ec430c628",
      "name": "User",
      "email": "guide@jonas.io"
    }
  ],
  "_id": "5c88fa8cf4afda39709c2951",
  "name": "The Forest Hiker",
  "duration": 5,
  "maxGroupSize": 25,
  "difficulty": "easy",
  "price": 397,
}
```

Tour şeması üzerinde oluşturulan **guides property'si** **User** şeması ile ilişki kurmak için kullanılan bir alandır. Tour şeması üzerinde bulunan **guides** özelliği turda görevli kullanıcıların **id** bilgilerini tutar. Guides ve id üzerinden kurulan ilişki ile turda görevli kullanıcı bilgileri alınabilir. Tour bilgileri üzerinde yapılan her **find** ile başlayan sorgularda Tour.guides üzerinde tutulan idler üzerinden **this.populate()** metodu işletilerek kullanıcıların diğer bilgileri getirilir.

Modelling Data and Advanced Mongoose

Review Schema

Sy: 3

reviewSchema

```
1 const reviewSchema = new mongoose.Schema(  
2   {  
3     review: {  
4       type: String,  
5       required: [true, 'Review can not be empty!'],  
6     },  
7     rating: {  
8       type: Number,  
9       min: 1,  
10      max: 5,  
11    },  
12    createdAt: {  
13      type: Date,  
14      default: Date.now,  
15    },  
16    tour: {  
17      type: mongoose.Schema.ObjectId,  
18      ref: 'Tour',  
19      required: [true, 'Review must belong to a tour'],  
20    },  
21    user: {  
22      type: mongoose.Schema.ObjectId,  
23      ref: 'User',  
24      required: [true, 'Review must belong to a user'],  
25    },  
26  },  
27  {  
28    toJSON: { virtuals: true },  
29    toObject: { virtuals: true },  
30  },  
31  });  
32  
33 reviewSchema.pre(/^find/, function (next) {  
34   this.populate({  
35     path: 'tour',  
36     select: 'name',  
37   }).populate({  
38     path: 'user',  
39     select: 'name photo',  
40   });  
41  
42   next();  
43 });
```

reviewController

```
1 exports.getAllReviews = catchAsync(async (req, res, next) => {  
2   const reviews = await Review.find();  
3  
4   res.status(200).json({  
5     status: 'success',  
6     results: this.getAllReviews.length,  
7     data: {  
8       reviews,  
9     },  
10  });  
11 });  
12  
13 exports.createReview = catchAsync(async (req, res, next) => {  
14   const newReview = await Review.create(req.body);  
15  
16   res.status(201).json({  
17     status: 'success',  
18     data: {  
19       review: newReview,  
20     },  
21   });  
22 });
```

Create New Review

HTTP Jonas Natours 2025 / Review / Create New Review

POST {{URL}} api/v1/reviews

Params Authorization Headers (11) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL

```
1 {  
2   "review": "Loved it!",  
3   "rating": 4,  
4   "tour": "5c88fa8cf4afda39709c2951",  
5   "user": "682da32af55f7ce393e30f41"  
6 }
```

Body Cookies (1) Headers (21) Test Results

JSON Preview Visualize

```
1 {  
2   "status": "success",  
3   "data": {  
4     "review": {  
5       "_id": "682da3baf55f7ce393e30f47",  
6       "review": "Loved it!",  
7       "rating": 4,  
8       "tour": "5c88fa8cf4afda39709c2951",  
9       "user": "682da32af55f7ce393e30f41",  
10      "createdAt": "2025-05-21T09:58:18.510Z",  
11      "__v": 0,  
12      "id": "682da3baf55f7ce393e30f47"  
13    }  
14  }  
15 }
```

Get Review

GET {{URL}} api/v1/reviews

Params Authorization Headers (9) Body Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies (1) Headers (21) Test Results

JSON Preview Visualize

```
31 {  
32   "_id": "682da3baf55f7ce393e30f47",  
33   "review": "Loved it!",  
34   "tour": {  
35     "guides": [  
36       {  
37         "role": "guide",  
38         "_id": "682cb8aaad40925ec430c628",  
39         "name": "User",  
40         "email": "guide@jonas.io"  
41       },  
42     ],  
43     "_id": "5c88fa8cf4afda39709c2951",  
44     "name": "The Forest Hiker",  
45     "durationWeeks": null,  
46     "id": "5c88fa8cf4afda39709c2951"  
47   },  
48   "user": {  
49     "_id": "682da32af55f7ce393e30f41",  
50     "name": "User"  
51   },  
52   "createdAt": "2025-05-21T09:58:18.510Z",  
53   "__v": 0,  
54   "id": "682da3baf55f7ce393e30f47"  
55 }
```

Turlar hakkında yapılan görüşleri **Review Schema** üzerinden veritabanına yazılır. Girilen bir **review** üzerinde **tourId** ve **userId** bulundurmak tour ve user dökümanlarının gereksiz büyümesini engeller. Gerektiğinde review üzerinde bulunan tourId ve userId ile ilgili dökümanların bilgileri **populate** metoduyla çekilebilir.

Modelling Data and Advanced Mongoose

Populate Fault!

Sy: 4

Get A Tour

```
43 {
44   "imageCover": "tour-1-cover.jpg",
45   "locations": [
46     {
47       "lat": 34.04,
48       "long": 15.04,
49       "order": 1
50     },
51     {
52       "lat": 35.04,
53       "long": 16.04,
54       "order": 2
55     }
56   ],
57   "slug": "the-forest-hiker",
58   "__v": 0,
59   "durationWeeks": 0.7142857142857143,
60   "reviews": [
61     {
62       "_id": "682da3baf55f7ce393e30f47",
63       "review": "Loved it!",
64       "tour": {
65         "id": "5c88fa8cf4afda39709c2951",
66         "name": "The Forest Hiker",
67         "durationWeeks": null,
68         "id": "5c88fa8cf4afda39709c2951"
69       },
70       "user": {
71         "_id": "682da32af55f7ce393e30f41",
72         "name": "User"
73       },
74       "createdAt": "2025-05-21T09:58:18.510Z",
75       "updatedAt": "2025-05-21T09:58:18.510Z"
76     }
77   ],
78   "created": "2025-05-21T09:58:18.510Z",
79   "updatedAt": "2025-05-21T09:58:18.510Z"
80 }
```

reviewSchema Find Middleware

```
33 reviewSchema.pre(/^find/, function (next) {
34   this.populate({
35     path: 'tour',
36     select: 'name',
37   }).populate({
38     path: 'user',
39     select: 'name photo',
40   });
41   next();
42 });
```

reviewSchema Middleware Fix

```
1 reviewSchema.pre(/^find/, function (next) {
2   this.populate({
3     path: 'user',
4     select: 'name photo',
5   });
6   next();
7 });
```

```
43 {
44   "imageCover": "tour-1-cover.jpg",
45   "locations": [
46     {
47       "lat": 34.04,
48       "long": 15.04,
49       "order": 1
50     },
51     {
52       "lat": 35.04,
53       "long": 16.04,
54       "order": 2
55     }
56   ],
57   "slug": "the-forest-hiker",
58   "__v": 0,
59   "durationWeeks": 0.7142857142857143,
60   "reviews": [
61     {
62       "_id": "682da3baf55f7ce393e30f47",
63       "review": "Loved it!",
64       "tour": {
65         "id": "5c88fa8cf4afda39709c2951",
66         "name": "The Forest Hiker",
67         "durationWeeks": null,
68         "id": "5c88fa8cf4afda39709c2951"
69       },
70       "user": {
71         "_id": "682da32af55f7ce393e30f41",
72         "name": "User"
73       },
74       "createdAt": "2025-05-21T09:58:18.510Z",
75       "updatedAt": "2025-05-21T09:58:18.510Z"
76     }
77   ],
78   "created": "2025-05-21T09:58:18.510Z",
79   "updatedAt": "2025-05-21T09:58:18.510Z"
80 }
```

tourSchema Middleware Fix (Virtual Property)

```
1 // Virtual populate
2 tourSchema.virtual('reviews', {
3   ref: 'Review',
4   foreignField: 'tour',
5   localField: '_id',
6 });
```

tourControl.getTour()

```
1 exports.getTour = catchAsync(async (req, res, next) => {
2   const tour = await Tour.findById(req.params.id).populate('reviews');
3
4   if (!tour) {
5     return next(new AppError('No tour found with that ID', 404));
6   }
7
8   res.status(200).json({
9     status: 'success',
10    data: {
11      tour,
12    },
13  });
14 });
```

Review Schema üzerinde **tour** ve **user** için populate işlemi uygulanarak bir **tour** dokümanı çekildiğinde, **tour** içinde bulunan **review** bilgisi üzerinden yeniden **tour** bilgisi gelir. Bilginin bu şekilde sürekli tekrar etmesi istenen bir durum değildir. Bu durumu düzeltmek için **reviewSchema** üzerinde bulunan **find middleware** üzerinde ki **tour populate** silinir. **tourSchema** üzerinde gereksiz alan barındırmamak için **review** dökümanına ait bir bilgi saklanmaz. Fakat **tour** üzerinde **review** bilgisini görüntülemek gerektiğinde bu durum çözmek için **virtual property** tanımlanabilir. **Virtual Property** hafızada çalışan sanal bir alan olduğu için bilgisayarda yer kaplamaz. **tourSchema** üzerinde **reviews** için **virtual property** hazırlandıktan sonra, **tourControl.getTour()** metodunda **populate('reviews')** uygulanır.

tourId **/tours/:tourId/reviews** üzerinden params ile
userId protect fonksiyonu ile logIn olan kullanıcı bilgileri
üzerinden alınmaktadır.

Modelling Data and Advanced Mongoose

Express Nested Routes

Sy: 6

Tour Routes

```
routes > JS tourRoutes.js > ...
27   authController.protect,
28   authController.restrictTo('admin', 'lead-guide'),
29   tourController.deleteTour,
30   );
31
32   // POST /tour/234fad4/reviews
33   // GET /tour/234fad4/reviews
34   // GET /tour/234fad4/reviews/94887fda
35
36   router
37     .route('/:tourId/reviews')
38     .post(
39       authController.protect,
40       authController.restrictTo('user'),
41       reviewController.createReview,
42     );
43
44   module.exports = router;
```

Review Routes

```
routes > JS reviewRoutes.js > ...
You, 23 hours ago | 1 author (You)
1   const express = require('express');
2   const reviewController = require('../controllers/reviewController');
3   const authController = require('../controllers/authController');
4
5   const router = express.Router();
6
7   router
8     .route('/')
9     .get(reviewController.getAllReviews)
10    .post(
11      authController.protect,
12      authController.restrictTo('user'),
13      reviewController.createReview,
14    );
15
16   module.exports = router;
```

Tour Routes Fix

```
1   const express = require('express');
2   const reviewRouter = require('../routes/reviewRoutes');
3
4   const router = express.Router();
5
6   router.use('/:tourId/reviews', reviewRouter);
7
8   module.exports = router;
```

Review Routes Fix

```
1   const express = require('express');
2   const reviewController = require('../controllers/reviewController');
3   const authController = require('../controllers/authController');
4
5   const router = express.Router({ mergeParams: true });
6
7   router
8     .route('/')
9     .get(reviewController.getAllReviews)
10    .post(
11      authController.protect,
12      authController.restrictTo('user'),
13      reviewController.createReview,
14    );
15
16   module.exports = router;
```

Create a review on tour

POST {{URL}} api/v1/tours/5c88fa8cf4afda39709c295d/reviews

Send

```
1 {
2   "rating": 3,
3   "review": "Was kind okay"
4 }
```

Body

201 Created - 19 ms - 1.19 KB

JSON

```
1 {
2   "status": "success",
3   "data": {
4     "review": {
5       "_id": "682e21ba6978d361a48525c6",
6       "rating": 3,
7       "review": "Was kind okay",
8       "tour": "5c88fa8cf4afda39709c295d",
9       "user": "682e168b51add1587411ee3e",
10      "createdAt": "2025-05-21T18:55:54.645Z",
11      "__v": 0,
12      "id": "682e21ba6978d361a48525c6"
13    }
14  }
15 }
```

Get tour

GET {{URL}} api/v1/tours/5c88fa8cf4afda39709c295d

Send

```
60   "coordinates": [
61     -122.408865,
62     37.787825
63   ],
64   "_id": "5c88fa8cf4afda39709c295e",
65   "description": "San Francisco",
66   "day": 5
67 },
68 ],
69 "slug": "the-city-wanderer",
70 "__v": 0,
71 "durationWeeks": 1.2857142857142858,
72 "reviews": [
73   {
74     "_id": "682e21ba6978d361a48525c6",
75     "rating": 3,
76     "review": "Was kind okay",
77     "tour": "5c88fa8cf4afda39709c295d",
78     "user": {
79       "_id": "682e168b51add1587411ee3e",
80       "name": "jonas"
81     }
82   }
83 ]
```

En üstte bulunan simple nested routes çözümü **tourRoutes** ve **reviewRoutes** üzerinde kod tekrarı oluşturur. Bu istenen bir durum değildir. Bu durumun çözümü için **router.use()** metodu kullanılır. **tourRoutes** üzerinde bulunan **router.use()** metodunun ilk parametresi route adresi, ikinci parametresi kullanılacak router metodudur(**reviewRouter**). **reviewRoutes** üzerinde **router** nesnesi oluşturulurken **express.Router()** metodu **mergeParams:true** değerini alarak router birleşimini gerçekleştirir.

Modelling Data and Advanced Mongoose

Create Reviews on Tour

Sy: 7

Post Review on Tour

```
1 //POST: {{URL}}api/v1/tours/5c88fa8cf4afda39709c295d/reviews
2 {
3   "rating": 4,
4   "review": "very good tour"
5 }
```

tourRoute

```
1 const express = require('express');
2 const reviewRouter = require('../routes/reviewRoute');
3
4 const router = express.Router();
5
6 router.use('/:tourId/reviews', reviewRouter);
7
8 module.exports = router;
```

reviewRoute

```
1 const express = require('express');
2 const reviewController = require('../controllers/reviewController');
3 const authController = require('../controllers/authController');
4
5 const router = express.Router({ mergeParams: true });
6
7 // POST /tour/234fad4/reviews
8 // GET /tour/234fad4/reviews
9 // POST /reviews
10
11 router
12   .route('/')
13   .get(reviewController.getAllReviews)
14   .post(
15     authController.protect,
16     authController.restrictTo('user'),
17     reviewController.createReview,
18   );
19
20 module.exports = router;
```

reviewController.createReview()

```
1 exports.createReview = catchAsync(async (req, res, next) => {
2   // Allow nested routers
3   if (!req.body.tour) req.body.tour = req.params.tourId;
4   if (!req.body.user) req.body.user = req.user.id;
5
6   const newReview = await Review.create(req.body);
7
8   res.status(201).json({
9     status: 'success',
10    data: {
11      review: newReview,
12    },
13  });
14 });
```

MongoDB

```
_id: ObjectId('68315308bfd2a56e2adbc387')
rating: 4
review: "very good tour"
tour: ObjectId('5c88fa8cf4afda39709c295d')
user: ObjectId('68315303bfd2a56e2adbc384')
createdAt: 2025-05-24T05:03:04.526+00:00
```

tours route'u üzerinden gönderilen reviews post'u öncelikle tourRoute üzerinden yakalanır ve burada bulunan router.use() metodu ile bu talep reviewRoute'una iletilir. tourRoute üzerinden gönderilen parametreler mergeParams:true parametresi ile birleştirilerek tourId değeri reviewRoute üzerinden alınır. reviewRoute üzerinden post işlemi öncelikle protect() metoduyla başlar. protect() metodu kullanıcının login olup olmadığına bakar. Eğer kullanıcı login olduysa user nesnesi protect() metodu üzerinden createReview() metoduna gönderilir.

createReview() metodu tourRoute() metodu üzerinden gelen parametre verisindeki tourId ve protect() metodundan gelen userId verisini alarak kullanıcı review bilgisini veritabanına yazar.

Modelling Data and Advanced Mongoose

Get Reviews on Tour

Sy: 8

{{URL}}api/v1/tours/5c88fa8cf4afda39709c295d/reviews

GET {{URL}} api/v1/tours/5c88fa8cf4afda39709c295d/reviews

Send

tourRoute

```
1 const express = require('express');
2 const reviewRouter = require('../routes/reviewRoute');
3
4 const router = express.Router();
5
6 router.use('/:tourId/reviews', reviewRouter);
7
8 module.exports = router;
```

reviewRoute

```
1 const express = require('express');
2 const reviewController = require('../controllers/reviewController');
3 const authController = require('../controllers/authController');
4
5 const router = express.Router({ mergeParams: true });
6
7 // POST /tour/234fad4/reviews
8 // GET /tour/234fad4/reviews
9 // POST /reviews
10
11 router
12   .route('/')
13   .get(reviewController.getAllReviews)
14   .post(
15     authController.protect,
16     authController.restrictTo('user'),
17     reviewController.createReview,
18   );
19
20 module.exports = router;
```

reviewController.getAllReviews()

```
1 exports.getAllReviews = catchAsync(async (req, res, next) => {
2   let filter = {};
3   if (req.params.tourId) filter = { tour: req.params.tourId };
4
5   // { tour: '5c88fa8cf4afda39709c295d' }
6
7   const reviews = await Review.find(filter);
8
9   res.status(200).json({
10     status: 'success',
11     results: reviews.length,
12     data: {
13       reviews,
14     },
15   });
16 });
```

reviewSchema

```
1 const reviewSchema = new mongoose.Schema(
2   {
3     review: {
4       type: String,
5       required: [true, 'Review can not be empty!'],
6     },
7     rating: {
8       type: Number,
9       min: 1,
10      max: 5,
11    },
12    tour: {
13      type: mongoose.Schema.ObjectId,
14      ref: 'Tour',
15      required: [true, 'Review must belong to a tour.'],
16    },
17    user: {
18      type: mongoose.Schema.ObjectId,
19      ref: 'User',
20      required: [true, 'Review must belong to a user'],
21    },
22  },
23  {
24    timestamps: true,
25  }
26 );
```

GET {{URL}} api/v1/tours/5c88fa8cf4afda39709c295d/reviews

Params Auth Headers (9) Body Scripts Settings Cookies

Body 200 OK 34 ms 1.46 KB Save Response

{ } JSON Preview Visualize

```
2 {
3   "status": "success",
4   // {{URL}}api/v1/tours/5c88fa8cf4afda39709c295d/reviews
5 }
6 {
7   "results": 2,
8   "reviews": [
9     {
10       "_id": "682e251ec3ecb67e7a17f1a6",
11       "rating": 3,
12       "review": "Was kind okay",
13       "tour": "5c88fa8cf4afda39709c295d",
14       "user": {
15         "_id": "682da32af55f7ce393e30f41",
16         "name": "User"
17       }
18     },
19     {
20       "_id": "68315308bfd2a56e2adbc387",
21       "rating": 4,
22       "review": "very good tour",
23       "tour": "5c88fa8cf4afda39709c295d",
24       "user": {
25         "_id": "68315303bfd2a56e2adbc384",
26         "name": "tuna"
27       }
28     }
29   ],
30 }
31
```

get() metoduna gelene kadar post() metodunda bulunan adımlar aynı şekildedir.

tourRoute üzerinden gelen tourId Review şeması üzerinden sorgu yapabilmek için filter isimli nesneye dönüştürülür ve find() metoduna argüman olarak gönderilir.

Bu işlemin sonucu olarak bir tour'a bağlı olan review bilgileri nested route yardımıyla yapılır.

Modelling Data and Advanced Mongoose

Handler Factory Function

Sy: 9

Önceki Delete Tour Fonksiyonu

```
1 exports.deleteTourOld = catchAsync(async (req, res, next) => {
2   const tour = await Tour.findByIdAndDelete(req.params.id);
3
4   if (!tour) {
5     return next(new AppError('No tour found with that ID', 404));
6   }
7
8   res.status(204).json({
9     status: 'success',
10    data: null,
11  });
12 });
```

```
1 // Handler Factory Function
2 exports.deleteTour = factory.deleteOne(Tour);
```

deleteOne()
Generic
Function

```
1 exports.deleteOne = (Model) =>
2   catchAsync(async (req, res, next) => {
3     const doc = await Model.findByIdAndDelete(req.params.id);
4
5     if (!doc) {
6       return next(new AppError('No document found with that ID', 404));
7     }
8
9     res.status(204).json({
10      status: 'success',
11      data: null,
12    });
13  });
```

Benzer kullanım alanına sahip control fonksiyonlarını her veri için farklı şekilde tanımlamak ve kullanmak yerine **Generic Fonksiyonları** (Handler Factory Function) kullanmak çok daha kullanışlıdır. Bir Tour dökümanını silmek için kullanılan deleteTour() fonksiyonu yerine model tipini argüman olarak alan ve bu argüman üzerinden işlemi yapan bir fonksiyonu döndüren (**Closures**) generic bir fonksiyon tanımlamak daha kullanışlıdır. Üstte tanımlanan deleteOne(Model) generic fonksiyonu sadece Tour verisi için değil User, Review... gibi bir çok model için kullanılabilir.

```
1 exports.getOne = (Model, popOptions) =>
2   catchAsync(async (req, res, next) => {
3     let query = Model.findById(req.params.id);
4     if (popOptions) query = query.populate(popOptions);
5     const doc = await query;
6
7     if (!doc) {
8       return next(new AppError('No document found with that ID', 404));
9     }
10
11     res.status(200).json({
12       status: 'success',
13       data: {
14         data: doc,
15       },
16     });
17   });
```

```
1 exports.getUser = factory.getOne(User);
```

getOne(Model, popOptions)

```
1 exports.getTour = factory.getOne(Tour, { path: 'reviews' });
```

getOne() fonksiyonunun kullanımı incelendiğinde hem bir **User** dökümanı hem de bir **Tour** dökümanı almak için kullanılabildiği görülmekte. Eğer bu fonksiyonun ile alınan dökümanın kendi içinde bağlantılı bulunan başka döküman bilgisini de almak isteniyorsa, parametre olarak populate edilecek modelin bilgisi argüman olarak **getOne()** fonksiyonuna verilebilir.

```
1 router.get(
2   '/me',
3   authController.protect,
4   userController.getMe,
5   userController.getUser,
6 );
```

{{URL}}api/v1/users/me

```
1 exports.protect = catchAsync(async (req, res, next) => {
2   // *** The code exists above.
3   // GRANT ACCESS TO PROTECTED ROUTE
4   req.user = currentUser;
5   next();
6 });
```

```
1 exports.getMe = (req, res, next) => {
2   req.params.id = req.user.id;
3   next();
4 };
```

```
1 exports.getUser = factory.getOne(User);
```

```
1 exports.getOne = (Model, popOptions) =>
2   catchAsync(async (req, res, next) => {
3     let query = Model.findById(req.params.id);
4     if (popOptions) query = query.populate(popOptions);
5     const doc = await query;
6
7     if (!doc) {
8       return next(new AppError('No document found with that ID', 404));
9     }
10
11     res.status(200).json({
12       status: 'success',
13       data: {
14         data: doc,
15       },
16     });
17   });
```

"users/me" route ile logın olmuş kullanıcının bilgileri alınır. protect() fonksiyonuyla kullanıcının logın olduğu kontrol edilir ve logın olan kullanıcı nesnesi **req.user** olarak **request**'e yüklenir. **getMe()** metoduyla **getOne()** fonksiyonun kullanacağı **req.params.id** parametresi hazırlanır ve **getOne()** fonksiyonuna yönlendirilir. Son olarak da **getOne()** ile ayrı bir fonksiyonun kullanılmadan **req.params.id** ile kullanıcı verisi veri tabanından alarak **response(res)** ile gönderilir.

Modelling Data and Advanced Mongoose

Handler Factory Function

Sy: 10

tourRoute

```
6 router.use('/:tourId/reviews', reviewRouter);
```

reviewRoute

```
5 const router = express.Router({ mergeParams: true });

1 router
2   .route('/')
3   .get(reviewController.getAllReviews)
4   .post(
5     authController.protect,
6     authController.restrictTo('user'),
7     reviewController.setTourUserIds,
8     reviewController.createReview,
9   );
```

Post Review on Tour

```
1 //POST: {{URL}}api/v1/tours/5c88fa8cf4afda39709c295d/reviews
2 {
3   "rating": 4,
4   "review": "very good tour"
5 }
```

tourId üzerinden bir **reviews** oluşturma **post request'i** öncelikle **tourRoute** üzerinden **reviewRouter'a** yönlendirilir. **reviewRouter** **mergeParams:true** ayarı ile kullanıcının gönderdiği parametreleri birleştirir ve **post request'ini** işletmeye başlar.

- **protect** middleware kullanıcı login kontrolü yapar ve login olmuş kullanıcı objesini **req(request'e)** yükler ve bir sonraki adıma **next** ile yollar.
- **restrictTo** middleware kullanıcı yetkilerini inceler. Sadece **user rolündeki** kullanıcı bir review tanımı oluşturabilir yetkisi tanımlandığından harici durumlar hata verdirilerek yetkisiz eylemler engellenir. Hata yoksa bir sonraki adıma geçilir.
- **setTourUserIds** middleware ile **tour router** üzerinden gelen parametre üzerinden gönderilen **tourId** ve **protect** middleware tarafından gönderilen **user.id** bilgisi alınır. Bir sonraki eylem olan **createReview()** metoduna geçilir.
- **createReview()** metodu en sonunda kullanıcı tarafından gönderilen review request'ini veri tabanına işler.

```
1 exports.protect = catchAsync(async (req, res, next) => {
2   // *** The code exists above.
3   // GRANT ACCESS TO PROTECTED ROUTE
4   req.user = currentUser;
5   next();
6 });
```

```
1 exports.restrictTo = (...roles) => {
2   return (req, res, next) => {
3     // roles ['admin', 'lead-guide']. role='user'
4     if (!roles.includes(req.user.role)) {
5       return next(
6         new AppError('You do not have permission to perform this action', 403),
7       );
8     }
9     next();
10  };
11 };
12
```

```
1 exports.setTourUserIds = (req, res, next) => {
2   // Allow nested routers
3   if (!req.body.tour) req.body.tour = req.params.tourId;
4   if (!req.body.user) req.body.user = req.user.id;
5   next();
6 };
```

```
1 exports.createReview = factory.createOne(Review);
```

```
1 exports.createOne = (Model) =>
2   catchAsync(async (req, res, next) => {
3     const doc = await Model.create(req.body);
4
5     res.status(201).json({
6       status: 'success',
7       data: {
8         data: doc,
9       },
10    });
11  });
```

createReview() fonksiyonu **createOne()** handler factory fonksiyonu tarafından oluşturulan kullanıcı review bilgisini veri tabanına yazan bir fonksiyondur.

createOne(Model) fonksiyonu argüman olarak aldığı **Model** sınıfına göre kullanıcı **req(request-istek)**'iğini veri tabanına yazacak fonksiyonu hazırlayıp döndüren javascript closures fonksiyonudur. **createOne()** fonksiyonu **User**, **Tour** gibi model sınıflarını da kullanarak bu modellere uygun fonksiyonları oluşturan kullanışlı jenerik bir fonksiyondur.

Modelling Data and Advanced Mongoose

Missing Auth and Indexes

Sy: 11

reviewRoutes

```
1 const express = require('express');
2 const reviewController = require('../controllers/reviewController');
3 const authController = require('../controllers/authController');
4
5 const router = express.Router({ mergeParams: true });
6
7 router.use(authController.protect);
8
9 router
10 .route('/')
11 .get(reviewController.getAllReviews)
12 .post(
13   authController.restrictTo('user'),
14   reviewController.setTourUserIds,
15   reviewController.createReview,
16 );
17
18 router
19 .route('/:id')
20 .get(reviewController.getReview)
21 .patch(
22   authController.restrictTo('user', 'admin'),
23   reviewController.updateReview,
24 )
25 .delete(
26   authController.restrictTo('user', 'admin'),
27   reviewController.deleteReview,
28 );
29
30 module.exports = router;
```

```
1 exports.protect = catchAsync(async (req, res, next) => {
2   // *** The code exists above.
3   // GRANT ACCESS TO PROTECTED ROUTE
4   req.user = currentUser;
5   next();
6 });
7
8 authController.protect()
```

userRoutes

```
1 const express = require('express');
2 const userController = require('../controllers/userController');
3 const authController = require('../controllers/authController');
4
5 const router = express.Router();
6
7 router.post('/signup', authController.signup);
8 router.post('/login', authController.login);
9 router.post('/forgotPassword', authController.forgotPassword);
10 router.patch('/resetPassword/:token', authController.resetPassword);
11
12 // Protect all routes after this middleware
13 router.use(authController.protect);
14
15 router.get('/me', userController.getMe, userController.getUser);
16 router.patch('/updateMyPassword', authController.updatePassword);
17 router.patch('/updateMe', userController.updateMe);
18 router.delete('/deleteMe', userController.deleteMe);
19
20 router.use(authController.restrictTo('admin'));
21
22 router
23 .route('/')
24 .get(userController.getAllUsers)
25 .post(userController.createUser);
26
27 router
28 .route('/:id')
29 .get(userController.getUser)
30 .patch(userController.updateUser)
31 .delete(userController.deleteUser);
32
33 module.exports = router;
```

Tüm router metodlarına tek tek kullanıcı girişi(Login) kontrolü yapan **authController.protect()** metodunu yerleştirmek yerine tüm router metodlarını kapsayacak bir **router.use(authController.protect)** kullanımı mümkündür. **reviewRoutes** üzerinden yapılan bir kullanıcı talebi düşünürse bu talep önce **router.use(authController.protect)** ile Login kontrolünden geçerek router işlemlerine geçer

Benzer bir kullanım şekli de kullanıcı yetkilerinde görülmektedir. Örneğin tüm kullanıcıların listesini almak isteyen bir istek talebi yapan kullanıcının önce Login olup olmadığı **protect()** metoduyla kontrol edilir, sonra kullanıcının admin olup olmadığı **router.use(authController.restrictTo('admin'))** işlemiyle kontrol edilir. Bu her iki kontrolü de geçen talep işletilir.

Bu işlem basamakları düşünülürken express.js kütüphanesinin *middleware çalışma mantığı unutulmamalıdır*. **authController.protect()** ve **authController.restrictTo()** fonksiyonları birer middleware fonksiyonlardır. Bu fonksiyonların çalışması bittiğinde **next()** metoduyla bir sonraki işleme geçilir. **next()** geçişiyle gerekiyorsa bilgiler bir sonraki fonksiyona aktarılabilir. Örneğin **protect()** metodunda **next()** işleminden önce **req.user = currentUser** ile req kullanıcı talebinin içine kullanıcı nesnesi eklenir ve bir sonra işletilecek fonksiyonun kullanımı için gönderilir.

getAll() Handler Factory Function

```
1 exports.getAll = (Model) =>
2   catchAsync(async (req, res, next) => {
3     // To allow for nested GET reviews on tour (hack)
4     let filter = {};
5     if (req.params.tourId) filter = { tour: req.params.tourId };
6
7     const features = new APIFeatures(Model.find(filter), req.query)
8       .filter()
9       .sort()
10      .limitFields()
11      .paginate();
12
13     const doc = await features.query.explain();
14
15     // SEND RESPONSE
16     res.status(200).json({
17       status: 'success',
18       results: doc.length,
19       data: {
20         data: doc,
21       },
22     });
23   });
```

Mongoose Schema index()

```
1 tourSchema.index({ price: 1, ratingsAverage: -1 });
2 tourSchema.index({ slug: 1 });
```

const doc = await features.query.explain();
.explain() metodu sorgu sonucuna **Model Schema** kullanımı ile ilgili bir çok bilgiyi ekler. Bu bilgiler yardımıyla yazılımda gerekli ayarlamalar yapılabilir.

Mongoose Schema içinde bulunan **.index()** metodu arama sorguları için mongoose veri tabanında index oluşturur. Bu indexler veri tabanının kullanım performansını ciddi olarak artırır.

Modelling Data and Advanced Mongoose

Calculating Average Rating on Tours (Statics) 1

Sy: 12

Tour Model (tourSchema)

```
1 const mongoose = require('mongoose');
2
3 const tourSchema = new mongoose.Schema({
4   ratingsAverage: {
5     type: Number,
6     default: 4.5,
7     min: [1, 'Rating must be above 1.0'],
8     max: [5, 'Rating must be below 5.0'],
9   },
10  ratingsQuantity: {
11    type: Number,
12    default: 0,
13  },
14  /** Codes... */
15 });
```

Create New Tour POST

```
POST {{URL}} api/v1/tours
1 {
2   "name": "New Test Tour2",
3   "duration": 1,
4   "maxGroupSize": 1,
5   "difficulty": "easy",
6   "price": 200,
7   "summary": "Test Tour",
8   "imageCover": "tour-3-cover.jpg"
9 }
10
11 "ratingsAverage": 4.5,
12 "ratingsQuantity": 0,
13 "images": [],
14 "createdAt": "2025-05-27T16:30:41.094Z",
15 "startDates": [],
16 "secretTour": false,
17 "guides": [],
18 "_id": "6835e90f6cefb6158c33680",
19 "name": "New Test Tour2",
```

User Login POST

```
POST {{URL}} api/v1/users/login
1 {
2   "email": "laura@example.com",
3   "password": "test1234"
4 }
5
6 "user": {
7   "role": "user",
8   "_id": "5c8a23c82f8fb814b56fa18d",
9   "name": "Laura Wilson",
10  "email": "laura@example.com",
```

Tour Model üzerinde bulunan **ratingAverage** (ortalama değerlendirme puanı) ve **ratingQuantity** (değerlendirme sayısı) alanları her girilen review rating değeriyle değişen alanlardır. Kullanıcılar her review rating girişi yaptığında bu alanlar hesaplanır ve review'in bağlı bulunduğu tour üzerindeki ratingAverage ve ratingsQuantity değerleri otomatik olarak hesaplanarak işlenir.

Review Model (reviewSchema)

```
1 const mongoose = require('mongoose');
2 const Tour = require('./tourModel');
3
4 const reviewSchema = new mongoose.Schema(
5   {
6     review: {
7       type: String,
8       required: [true, 'Review can not be empty!'],
9     },
10    rating: {
11      type: Number,
12      min: 1,
13      max: 5,
14    },
15    createdAt: {
16      type: Date,
17      default: Date.now,
18    },
19    tour: {
20      type: mongoose.Schema.ObjectId,
21      ref: 'Tour',
22      required: [true, 'Review must belong to a tour.'],
23    },
24    user: {
25      type: mongoose.Schema.ObjectId,
26      ref: 'User',
27      required: [true, 'Review must belong to a user'],
28    },
29  },
```

Review Model Statics calcAverageRating()

```
1 reviewSchema.statics.calcAverageRating = async function (tourId) {
2   const stats = await this.aggregate([
3     {
4       $match: { tour: tourId },
5     },
6     {
7       $group: {
8         _id: '$tour',
9         nRating: { $sum: 1 },
10        avgRating: { $avg: '$rating' },
11      },
12    },
13  ]);
14  console.log(stats);
15
16  if (stats.length > 0) {
17    await Tour.findByIdAndUpdate(tourId, {
18      ratingsQuantity: stats[0].nRating,
19      ratingsAverage: stats[0].avgRating,
20    });
21  } else {
22    await Tour.findByIdAndUpdate(tourId, {
23      ratingsQuantity: 0,
24      ratingsAverage: 4.5,
25    });
26  }
27 };
```

Tour model üzerinde bulunan **ratingAverage** ve **ratingQuantity** alanlarının varsayılan değerleri sırasıyla 4.5 ve 0 olarak tanımlanmıştır. Test için oluşturulan tour üzerinde bu değerler görülmekte. Reviw verilerini sadece **user** tipinde kullanıcılar oluşturabilmektedir. ratingQuantity alanı tour üzerinde bulunan toplam review sayısıdır, ratingAverage ise verilen **rating** puanlarının ortalamasıdır. Tour üzerine bu değerler **calcAverageRating() static metodu** yardımıyla her review girildiğinde otomatik olarak hesaplanarak işlenir. calcAverageRating() metodu review verileri üzerinde gerekli hesaplamaları yaptıktan sonra sonuçları **Tour** modeli kullanarak veritabanında bulunan Tour dökümanına findByIdAndUpdate() metodu yardımıyla yazar.

Modelling Data and Advanced Mongoose

Calculating Average Rating on Tours (Statics) 2

Sy: 13

Review Model Statics calcAverageRating()

```
1 reviewSchema.statics.calcAverageRating = async function (tourId) {
2   const stats = await this.aggregate([
3     {
4       $match: { tour: tourId },
5     },
6     {
7       $group: {
8         _id: '$tour',
9         nRating: { $sum: 1 },
10        avgRating: { $avg: '$rating' },
11      },
12    },
13  ]);
14  console.log(stats);
15
16  if (stats.length > 0) {
17    await Tour.findByIdAndUpdate(tourId, {
18      ratingsQuantity: stats[0].nRating,
19      ratingsAverage: stats[0].avgRating,
20    });
21  } else {
22    await Tour.findByIdAndUpdate(tourId, {
23      ratingsQuantity: 0,
24      ratingsAverage: 4.5,
25    });
26  }
27 };
```

Review Model Statics calcAverageRating()

```
1 // save
2 // findByIdAndUpdate
3 // findByIdAndDelete
4 reviewSchema.post(/save|^findOneAnd/, async (doc, next) => {
5   if (doc) {
6     await doc.constructor.calcAverageRating(doc.tour);
7   }
8   next();
9 });
```

Test Operations

```
1 Save
2 1 "_id": "", "rating": 5, "review": "COOL", 5
3 2 "_id": "", "rating": 1, "review": "Teerrible", 3
4 3 "_id": "", "rating": 5, "review": "AMAZING", 3,67
5 4 "_id": "", "rating": 5, "review": "Great", 4
6 5 "_id": "", "rating": 3, "review": "OK", 3,8
7 Update & Delete
8 5 "_id": "", "rating": 4, "review": "AMAZING", 3,6
9 5 "_id": "", "rating": 3, "review": "COOL", 3,4
10 4 "_id": "", Deleted 4
11 3 "_id": "", Deleted 4
12 2 "_id": "", Deleted 4
13 1 "_id": "", Deleted 3
14 0 "_id": "", Deleted 4.5
```

Post First Review

```
POST {{URL}} api/v1/tours/6835e90f6cefb6158c33680/reviews
1 {
2   "rating": 5,
3   "review": "COOL"
4 }
5 "data": {
6   "data": {
7     "_id": "6836c5d7c7d0d51425cd84d0",
8     "rating": 5,
9     "review": "COOL",
10    "tour": "6835e90f6cefb6158c33680",
11    "user": "5c8a23c82f8fb814b56fa18d",
12    "createdAt": "2025-05-28T08:14:15.077Z",
13    "__v": 0,
14    "id": "6836c5d7c7d0d51425cd84d0"
```

Get Tour After First Review Post

```
GET {{URL}} api/v1/tours/6835e90f6cefb6158c33680
4 "data": {
5   "startLocation": {
6     "type": "Point",
7     "coordinates": []
8   },
9   "ratingsAverage": 5,
10  "ratingsQuantity": 1,
11  "images": [],
12  "startDates": [],
13  "secretTour": false,
14  "guides": [],
15  "_id": "6835e90f6cefb6158c33680",
16  "name": "New Test Tour2",
```

Get Tour After Last Review Post

```
GET {{URL}} api/v1/tours/6835e90f6cefb6158c33680
4 "data": {
5   "startLocation": {
6     "type": "Point",
7     "coordinates": []
8   },
9   "ratingsAverage": 3.8,
10  "ratingsQuantity": 5,
11  "images": [],
12  "startDates": [],
13  "secretTour": false,
14  "guides": [],
15  "_id": "6835e90f6cefb6158c33680",
16  "name": "New Test Tour2",
```

Get Tour Data From DBAfter Last Review Post

```
_id: ObjectId('6835e90f6cefb6158c33680')
ratingsAverage: 3.8
ratingsQuantity: 5
name: "New Test Tour2"
```

Review Model üzerinde calcAverageRating() metodunu tanımlamak tour değeri üzerinde gerekli hesaplamayı yapmak için yeterli değildir. calcAverageRating() metodunun tetiklenmesi için review üzerinde yapılan her işlemde tetiklenecek .post() metodu tanımlanır. .post() metodu /save|^findOneAnd/ regular expartion ifadesi ile review üzerinde yapılan **her oluşturma, güncelleme ve silme** işleminde tetiklenir ve çalıştırılır. Bu sayfada 1 tour üzerinde oluşturulan 5 review işleminin sonucunda ratingAverage ve ratingQuantity değerlerinde meydana gelen değişimler görülmektedir.

Modelling Data and Advanced Mongoose

Calculating Average Rating on Tours (Statics) 3

Sy: 14

Test Operations

1	Save
2	1 "_id": "", "rating": 5, "review": "COOL", 5
3	2 "_id": "", "rating": 1, "review": "Terrible", 3
4	3 "_id": "", "rating": 5, "review": "AMAZING", 3,67
5	4 "_id": "", "rating": 5, "review": "Great", 4
6	5 "_id": "", "rating": 3, "review": "OK", 3,8
7	Update & Delete
8	5 "_id": "", "rating": 4, "review": "AMAZING", 3,6
9	5 "_id": "", "rating": 3, "review": "COOL", 3,4
10	4 "_id": "", Deleted 4
11	3 "_id": "", Deleted 4
12	2 "_id": "", Deleted 4
13	1 "_id": "", Deleted 3
14	0 "_id": "", Deleted 4.5

Patch a Review Rating

PATCH	{{URL}} api/v1/reviews/6836c726c7d0d51425cd84e0
1	{
2	"rating": 4
3	}
4	"data": {
5	"_id": "6836c726c7d0d51425cd84e0",
6	"rating": 4,
7	"review": "AMAZING",
8	"tour": "6835e90f6cefbb6158c33680",
9	"user": {
10	"_id": "5c8a23c82f8fb814b56fa18d",
11	"name": "Laura Wilson",
12	"photo": "user-14.jpg"
13	},
14	"createdAt": "2025-05-28T08:19:50.380Z",
15	"__v": 0,
16	"id": "6836c726c7d0d51425cd84e0"

Get Tour After Patch a Review Post

GET	{{URL}} api/v1/tours/6835e90f6cefbb6158c33680
4	"data": {
5	"startLocation": {
6	"type": "Point",
7	"coordinates": []
8	},
9	"ratingsAverage": 3.6,
10	"ratingsQuantity": 5,
11	"images": [],
12	"startDates": [],
13	"secretTour": false,
14	"guides": [],
15	"_id": "6835e90f6cefbb6158c33680",
16	"name": "New Test Tour2",
17	"duration": 1,
18	"maxGroupSize": 1,
19	"difficulty": "easy",
20	"price": 200,
21	"summary": "Test Tour",
22	"imageCover": "tour-3-cover.jpg",
23	"location": {}

Bir review rating değeri güncelleniğinde tour üzerinde bulunan ratingAverage değerinin değiştiği görülmekte.

Property Value Rounding

1	const mongoose = require('mongoose');
2	
3	const tourSchema = new mongoose.Schema({
4	ratingsAverage: {
5	type: Number,
6	default: 4.5,
7	min: [1, 'Rating must be above 1.0'],
8	max: [5, 'Rating must be below 5.0'],
9	},
1	ratingsAverage: {
2	type: Number,
3	default: 4.5,
4	min: [1, 'Rating must be above 1.0'],
5	max: [5, 'Rating must be below 5.0'],
6	set: (val) => Math.round(val * 10) / 10,
7	},

set özelliği tanımlanan fonksiyon yardımı ile değerler üzerinde matematiksel yapılmasını sağlar. Üstteki işlemde ratingAverage değeri virgülden sonra bir basamak olacak şekilde otomatik bir şekilde yuvarlanır. Ör: 4,7 gibi.

One review can only created a user

```
1 reviewSchema.index({ tour: 1, user: 1 }, { unique: true });
```

Bir review'in sadece bir kullanıcı tarafından oluşturulması gerekir. Bu işlem Review Model üzerinde yapılacak index tanımı yapılır.